# Методические указания к лабораторной работе

**Автор: доцент НИТПУ Друки А.А.**

## Распознавание изображений с использованием архитектуры ResNet

Данная лабораторная работа посвящена распознаванию изображений из выборки CIFAR10 с помощью модели нейронной сети ResNet.

**Цель работы:** получить навыки распознаванию изображений с помощью модели нейронной сети ResNet.

**Задачи:**

1. Ознакомиться с работой сверточных нейронных сетей в библиотеке Keras;
2. Научиться применять модель сверточной нейронной сети ResNet для распознавания изображений.

## Модель сверточной нейронной сети Resnet

**Resnet** (residual network - остаточная сеть) – это свёрточная нейронная сеть, которая была представлена в 2015 году. В ней заложены идеи обходных путей в нейронных сетях: подаём выходные данные двух успешных свёрточных слоёв и обходим входные данные для следующего слоя.

Для распознавания сложных образов на изображениях бывает недостаточно нейросетей с малым количеством слоев, поэтому возникает потребность наращивания слоев, тем самым усложняя нейронную сеть и делая её более громоздкой. Обучение ИНС с большим числом слоев как правило проходит неравномерно. Последние слои (те, что ближе к выходу) будут обучаться быстрее начальных. В результате, ошибка на выходе будет стремиться к нулю и, как следствие, градиенты также будут принимать малые значения. Если мы умножаем эти значения (величины меньше 0) друг на друга, то получаются очень маленькие числа. Таким образом происходит постепенное затухание градиента. Хорошо обученные последние слои «блокируют» обучение начальных слоев и алгоритм обратного распространения, фактически, застревает на определенном уровне. Все это известно под названием проблема затухающих градиентов.

Чтобы преодолеть эту проблему, была разработана глубокая «остаточная» структура обучения. Связи между слоями идут через обходные пути минуя несколько слоев и их выходы суммируются с выходами других слоев, которые идут стандартным прямым путем (рисунок 1).

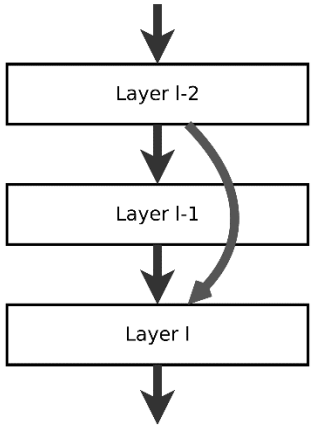



Рисунок 1. Организация слоев в ResNet

ResNet позволяет относительно легко увеличить точность благодаря увеличению глубины, чего с другими сетями добиться сложнее.

## Ход работы

### 1. Загрузка данных

В качестве данных для обучения и тестирования будет использоваться набор данных CIFAR-10 (Канадский институт перспективных исследований) – это один из наиболее широко используемых наборов данных для исследований в области машинного обучения. Набор данных CIFAR-10 содержит 60000 цветных изображений размера 32x32 пикселей. Изображения содержат 10 различных классов объектов: самолеты, автомобили, птицы, кошки, олени, собаки, лягушки, лошади, корабли и грузовики. По 6000 изображений на каждый класс. Поскольку изображения в CIFAR-10 имеют низкое разрешение (32x32), этот набор данных может позволить исследователям быстро опробовать различные алгоритмы, чтобы увидеть эффективность их работы. На рисунке 2 представлены примеры изображений CIFAR-10.

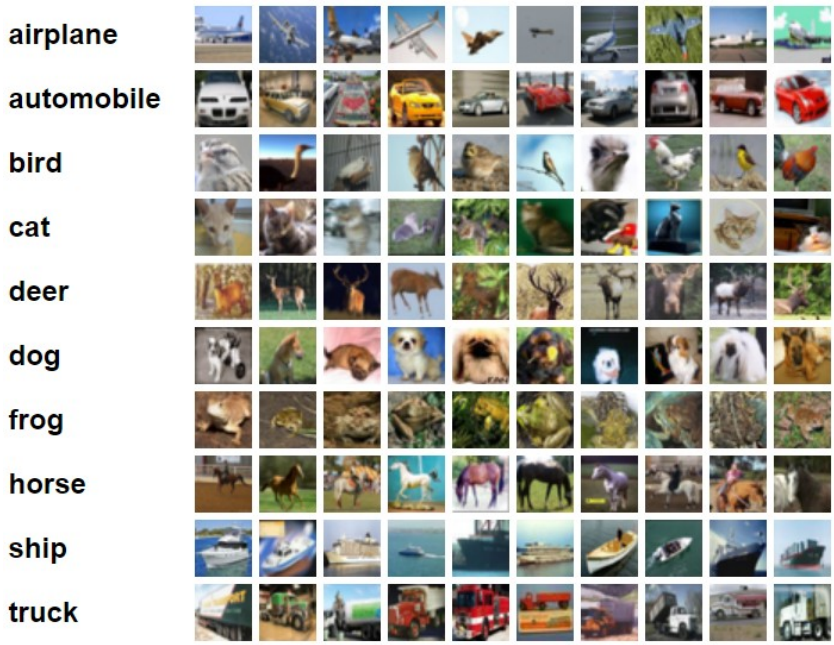



Рисунок 2. Примеры изображений CIFAR-10

Инструменты для загрузки CIFAR-10 уже встроены в Keras. Для начала работы необходимо открыть Google Colab и выполнить следующий блок кода:

```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np
```

**NumPy** – библиотека языка Python, добавляющая поддержку больших многомерных массивов и матриц.

**Matplotlib** – пользовательская библиотека для построения графиков на языке python.

Далее следует загрузить изображения CIFAR-10 в 4 переменные: X_train, Y_train, X_test, Y_test.

```
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

**X_train** – Исходные изображения из обучающей выборки данных.
**Y_train** – Выходные данные из обучающей выборки данных.
**X_test** – Исходные изображения из тестовой выборки данных.
**Y_test** – Выходные данные из тестовой выборки.

Чтобы ускорить процесс обучения нейронных сетей вы можете переключить среду выполнения с процессора (CPU) на графический процессор (GPU). Вычисления будут производиться с помощью GPU, что позволит ускорить обучение нейронной сети в несколько раз.

Для этого выберите пункт «Среда выполнения» -> «Сменить среду выполнения» (рисунок 3).

### 2. Вывод загруженных изображений на экран

Для того чтобы проверить корректность загрузки наших данных выведем на экран несколько загруженных изображений из датасета. Также выведем класс каждой картинки в виде подписи снизу.

```
plt.figure(figsize=(6,6))
for i in range(16):
    plt.subplot(4,4,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(x_train[i], cmap=plt.cm.binary)
    plt.xlabel(y_train[i])
```

На рисунке 5 представлены выведенные на экран изображения из обучающей выборки.




Рисунок 5. Вывод загруженных изображений

### 3. Предобработка данных

Перед обучением нейронной сети проведем предварительную обработку данных, используя следующий блок кода:

```
x_train = x_train/np.max(x_train)
x_test = x_test/np.max(x_test)
```

Как видно в коде происходит деление данных на значение np.max(x_train) и np.max(x_test). Так обозначены максимально возможные значения пикселей и равны они 255, так как значения пикселей на изображениях имеют значения в диапазоне [0; 255]. В результате деления элементов массивов на значение 255, значения элементов будут приведены к интервалу [0; 1].

На выходе сети будут 10 нейронов, соответствующие 10 класса объектов и принимающие значения: 0 – False; 1 – True. Для этого выполняется следующая функция:

```
y_train_cat = keras.utils.to_categorical(y_train, 10)
y_test_cat = keras.utils.to_categorical(y_test, 10)
```

**to_categorical** - преобразует вектор класса (целые числа) в двоичную матрицу.

**Y_train_cat** - предобработанные исходные изображения обучающей выборки данных.

**Y_test_cat** - предобработанные данные из обучающей выборки.

### 4. Реализация топологии нейронной сети ResNet

На рисунке 6 представлен небольшой фрагмент структуры ResNet. Архитектура ResNet содержит 150 слоев. Как видно по рисунку она имеет не последовательную структуру. Обходные связи позволяют «перепрыгивать» некоторые слои. Поэтому, для описания такой архитектуры воспользуемся функциональным подходом. На вход сети подаются изображения размером 32x32 пикселя с тремя цветовыми каналами (RGB). В выходном слое расположено 10 нейронов, что соответствует количеству классов распознаваемых объектов. В процессе работы входные данные распределяются по слоям НС, при этом выполняются операции: умножение на весовые коэффициенты, суммирование данных, вычисление функции активации. В выходном слое происходит классификация путём активации соответствующего нейрона.

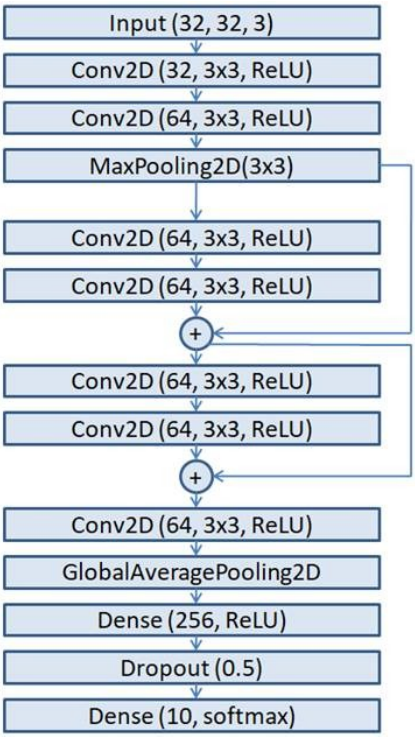



Рисунок 6. Пример модели нейронной сети

Первый блок из последовательных слоев можно представить, следующим образом:

```
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import *


.
.
Допишите код
.
.
```

Его выход связываем со следующим блоком:

```
.
.
Допишите код
.
.
```

И формируем суммарный сигнал с выходов x и block_1_output:

```
.
.
Допишите код
.
.
```

Как видите, в Keras имеется множество вспомогательных слоев. В частности, с помощью слоя add мы описываем операцию суммирования двух входов.

Далее, суммарный выход block_2_output подаем на вход следующего блока и по аналогии формируем суммарный слой для выходов x и block_2_output:

```
.
```

```
.
Допишите код
.
.
```

Наконец, суммарный тензор block_3_output подаем на последний блок и формируем на выходе классы изображений:

```
.
.
Допишите код
.
.
```

**Conv2D** – создание сверточного слоя со следующими параметрами: количество плоскостей (фильтров), размер ядра свертки, **input_shape** – формат входных данных (количество пикселей изображения, количество цветовых каналов), **activation** – функция активации, **padding** – параметр "same" означает не изменять размер изображения после его обработки сверточным слоем. Изменяя параметры слоя **Conv2D** можно повысить качество работы нейронной сети. Данный слой с помощью ядра свертки выполняет скалярное произведение входных данных и весов.

**MaxPooling2D** – создание слоя Max-pooling. Окном 2x2 сканируется предыдущий слой, выбирает в каждой такой области максимальный элемент и записывает его в текущий слой. Уменьшает размер входного слоя в 2 раза.

**ReLU** – функция активации, выполняющая простое нелинейное преобразование поданных на вход данных (x). Возвращает x, если x > 0 и 0 в противном случае. Отличается высокой скоростью вычисления.

В последнем блоке используется слой **GlobalAveragePooling2D**, который вычисляет среднее арифметическое по всем каналам.

А перед выходным слоем используется **Dropout()** для увеличения обобщающей способности сети. Параметр 0,5 означает, что при обучении будет использоваться половина нейронов из 256 предыдущего полносвязного слоя. Этим самым, мы пытаемся снизить (а в идеале исключить) эффект переобучения.

Далее идут стандартные операции для инициализации модели:

```
.
.
Допишите код
.
.
```

**5. Обучение нейронной сети**

Для обучения нейронной сети необходимо вызвать метод **fit()**, используя следующий блок кода:

```
.
.
Допишите код
.
.
```

### 6. Тестирование нейронной сети

Для тестирования нейронной сети необходимо запустить следующий блок кода:

```
.
.
Допишите код
.
.
```

### 7. Результат работы нейронной сети

Выполните следующий блок кода:

```
.
.
Допишите код
.
.
```

**y_pred** – выполняет предсказания модели на тестовых данных. Будет содержать вероятности принадлежности каждого примера к каждому классу.

**y_pred_classes** – преобразует вероятности в предсказанные классы. Для каждого примера выбирается класс с максимальной вероятностью.

**y_true** – преобразует one-hot метки в числовые классы.

Выведем несколько случайных изображений из тестового датасета. Так же сделаем сверху подписи: зеленый цвет – если нейрсоеть дала правильный результат; красный цвет – если нейросеть ошиблась (рисунок 7).

```
plt.figure(figsize=(13, 8))
random_indices = np.random.choice(len(x_test), size=15, replace=False)

for i, idx in enumerate(random_indices):
    plt.subplot(3, 5, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_test[idx].reshape(32, 32, 3), cmap=plt.cm.binary)
    predicted_label = y_pred_classes[idx]
    true_label = y_true[idx]
    color = 'green' if predicted_label == true_label else 'red'
    plt.title(f'Pred: {predicted_label}\nTrue: {true_label}', color=color,
fontsize=12)

plt.tight_layout()
plt.show()
```

На рисунке 7 представлен результат распознавания объектов на изображениях нейронной сетью ResNet.




Рисунок 7. Результат работы нейронной сети

## Вывод

В результате выполнения данной лабораторной работы Вы получили навыки распознавания объектов на изображениях из выборки CIFAR-10 с использованием библиотеки машинного обучения Keras. Также научились реализовывать топологию сверточной нейронной сети ResNet, обучать её и тестировать.

## Критерии оценки

Максимальное число баллов, которое можно получить по лабораторной работе – 8 баллов.

Минимальное число баллов, при котором задание засчитывается – 5 баллов.

**Выполнение задания**

Выполнено меньше половины задания - 0 баллов.

Большей частью выполнено - 2 балл.

Выполнено полностью - 4 балла.

**Качество отчета**

Сильные огрехи в оформлении или удовлетворяет менее половины требований к отчету - 0 баллов.

Хорошее оформление. Удовлетворяет большей части требований - 2 балл.

Соответствует всем требованиям - 4 балла.